

About type inference

Sébastien WILMET

December 2023

Abstract

In my opinion, type inference for variable declarations should be used only sparingly, when the type of the variable is already visible (and quite long to write) on the right-hand side of the assignment. Writing the types of variables explicitly enhance code comprehension.

Introduction

Some programming languages support *type inference*¹ for variable declarations with a keyword like `var` (used by C#, Vala, modern Java, ...), `let` (Rust²), `auto` (modern C++) or nothing at all (Python), to give just a few examples.

It seems that all newly-created languages tend to support type inference, like in Rust where it is advertised as “high-level ergonomics” because the developer needs to write less code.

For older languages like Java, type inference is introduced in new versions of the language. So it is something more and more ubiquitous.

Examples

Examples where the code is still easily comprehensible:

```
var n_days_of_rain = 10;  
var list = new ArrayList<String>();
```

An example where it becomes problematic:

```
var items = function_call();
```

Here, we don't know the type of the `items` variable. We need to look at the documentation of `function_call()` to know what kind of values it returns.

Text editor and IDE support — or not

A typical feature of text editors and IDEs for such programming languages is that when you point your mouse on the keyword (`var` in the above examples) or the variable name, it shows a “tooltip” (a bubble of text) with the inferred type.

On the other hand, if you want to learn the programming language by reading a (plain old printed) book, then you're out of luck.

¹https://en.wikipedia.org/wiki/Type_inference

²In Rust, all variable declarations actually need to use the `let` keyword, and explicit type information is added optionally.

Or, in a distant future, if the programming language in question has become old-fashioned, the tooling around it might not have aged well and you need to use a simple text editor to read the legacy source code.

Or, another context where this tooling feature may be absent but is crucial: patch review.

Why bother with knowing the type

Knowing the type of a variable is useful for at least two purposes:

1. To know what the variable represents (the object).
2. To know what operations can be used with it (operators and function/method calls).

Small rant about Python code that looks clean

A prominent reason why Python code looks cleaner than — say — C code (beginners are less afraid by it), is justly because all the variable types are omitted, so the code is more succinct. But it doesn't mean that it's easier to actually understand and modify it.

For a few hundreds lines of code (a script), it doesn't matter much. For a big codebase that needs to be maintained in the long run, there is a bigger difference.

Note that recent Python versions support what is called “type hints”, to add type information. If the type hints feature has been added to the language, it can only mean something.

As an anecdote, I know someone who was learning programming with Python as his first language, and when he asked me a question and I saw his code, I realized that he used type casts for almost *all* variable assignments. I then talked about Java, but unfortunately it doesn't have all the libraries for scientific-oriented tasks.

Compilers anyway need to implement it internally

For statically-typed languages, and for type safety, the compiler anyway needs to implement type inference. To check that the types are correct and to emit error messages otherwise.

So once it is implemented in the compiler, it is easy to expose the feature in the language syntax. For me, it is an explanation on why type inference is more and more present.

Conclusion

To conclude, I may be a bit old-school, but I prefer explicitly-typed programming languages like C and non-modern Java code.

When the code is too compact, it can be hard to follow. With more redundancy, there are less chances for logic errors (with a good compiler, that being said).